

Filtrování v JOIN a OUTER JOIN v PostgreSQL

Příklady používají schema `public` a plně kvalifikované názvy tabulek.

Všechny příklady jsou pouze dotazy `SELECT` a nemění data.

1. Základní rozdíl mezi `INNER JOIN` a `LEFT JOIN`

`INNER JOIN` vrací pouze řádky, pro které existuje odpovídající řádek na obou stranách spojení.

`LEFT JOIN` vrací všechny řádky z levé tabulky. Pokud odpovídající řádek v pravé tabulce neexistuje, sloupce z pravé tabulky mají hodnotu `NULL`.

```
SELECT
    film.film_id,
    film.title,
    language.name AS language_name
FROM public.film AS film
INNER JOIN public.language AS language
    ON language.language_id = film.original_language_id
ORDER BY film.film_id
LIMIT 10;
```

```
SELECT
    film.film_id,
    film.title,
    language.name AS language_name
FROM public.film AS film
LEFT JOIN public.language AS language
    ON language.language_id = film.original_language_id
ORDER BY film.film_id
LIMIT 10;
```

U druhého dotazu zůstanou zachovány i filmy bez vyplněného `original_language_id`.

2. Filtrování v `ON` u `LEFT JOIN`

Podmínka v části `ON` určuje, které řádky se k sobě smí připojit. Neurčuje, které řádky z levé tabulky se mají vyřadit.

```
SELECT
    film.film_id,
    film.title,
    language.name AS original_language
FROM public.film AS film
```

```
LEFT JOIN public.language AS language
  ON language.language_id = film.original_language_id
 AND language.name = 'English'
ORDER BY film.film_id
LIMIT 20;
```

Spojení zachová všechny filmy, ale **LIMIT** zobrazí jen prvních 20. Pokud původní jazyk filmu není **English**, pravá strana spojení bude **NULL**.

Tento způsob je vhodný, pokud chceme zachovat všechny záznamy z levé tabulky a pouze omezit, jaké záznamy se k nim připojí.

3. Stejný filtr v **WHERE** u **LEFT JOIN**

Pokud stejnou podmínku přesuneme do **WHERE**, nejprve vznikne výsledek **LEFT JOIN**, ale potom se odstraní řádky, kde podmínka není splněna. Řádky s **NULL** na pravé straně proto zmizí.

```
SELECT
  film.film_id,
  film.title,
  language.name AS original_language
FROM public.film AS film
LEFT JOIN public.language AS language
  ON language.language_id = film.original_language_id
WHERE language.name = 'English'
ORDER BY film.film_id
LIMIT 20;
```

Tento dotaz se z hlediska výsledku chová podobně jako **INNER JOIN** s filtrem na jazyk **English**. **WHERE language.name = 'English'** totiž nepropustí hodnotu **NULL**.

Porovnání počtů

Následující dotaz ukáže rozdíl mezi filtrem v **ON** a filtrem ve **WHERE**.

```
SELECT
  (SELECT COUNT(*)
   FROM public.film AS film
   LEFT JOIN public.language AS language
     ON language.language_id = film.original_language_id
    AND language.name = 'English') AS rows_with_filter_in_on,
  (SELECT COUNT(*)
   FROM public.film AS film
   LEFT JOIN public.language AS language
     ON language.language_id = film.original_language_id
   WHERE language.name = 'English') AS rows_with_filter_in_where,
  (SELECT COUNT(*) FROM public.film) AS all_films;
```

První počet odpovídá počtu všech filmů. Druhý počet obsahuje pouze filmy s odpovídajícím jazykem **English**.

4. Jak zachovat řádky bez shody při filtrování ve **WHERE**

Někdy chceme použít **WHERE**, ale současně zachovat i řádky bez odpovídající hodnoty. V takovém případě je potřeba výslovně přidat podmínku **IS NULL**.

```
SELECT
    film.film_id,
    film.title,
    language.name AS original_language
FROM public.film AS film
LEFT JOIN public.language AS language
    ON language.language_id = film.original_language_id
WHERE language.name = 'English'
    OR language.name IS NULL
ORDER BY film.film_id
LIMIT 20;
```

Výsledek obsahuje filmy s původním jazykem **English** i filmy, u kterých žádný původní jazyk není uveden. Filmy s jiným známým jazykem se nevrátí.

5. Filtr levé tabulky: **ON** versus **WHERE**

Když filtruje sloupec z levé tabulky, rozdíl je ještě názornější.

Filtr v **ON** zachová i filmy, které podmínku nesplní:

```
SELECT
    film.film_id,
    film.title,
    language.name AS original_language
FROM public.film AS film
LEFT JOIN public.language AS language
    ON language.language_id = film.original_language_id
    AND film.film_id <= 10
ORDER BY film.film_id
LIMIT 20;
```

Filtr ve **WHERE** vyřadí filmy s **film_id** větším než 10:

```
SELECT
    film.film_id,
    film.title,
    language.name AS original_language
FROM public.film AS film
LEFT JOIN public.language AS language
    ON language.language_id = film.original_language_id
```

```
WHERE film.film_id <= 10
ORDER BY film.film_id
LIMIT 20;
```

Praktické pravidlo: podmínka v **ON** ovlivňuje spojení, zatímco podmínka ve **WHERE** filtruje celý výsledek po spojení.

6. **RIGHT JOIN** jako opačný pohled

RIGHT JOIN zachová všechny řádky z pravé tabulky. Je možné jej přepsat jako **LEFT JOIN** prohozením pořadí tabulek.

```
SELECT
    film.film_id,
    film.title,
    language.name AS language_name
FROM public.language AS language
RIGHT JOIN public.film AS film
    ON film.language_id = language.language_id
ORDER BY film.film_id
LIMIT 10;
```

Tento dotaz zachová všechny filmy, protože **film** je pravá tabulka. Přehlednější ekvivalent je:

```
SELECT
    film.film_id,
    film.title,
    language.name AS language_name
FROM public.film AS film
LEFT JOIN public.language AS language
    ON film.language_id = language.language_id
ORDER BY film.film_id
LIMIT 10;
```

Filtr na **language.name** v **WHERE** by opět odstranil filmy, pro které se jazyk nepřipojil.

7. **FULL OUTER JOIN**

FULL OUTER JOIN zachová všechny řádky z obou tabulek. Pokud pro řádek neexistuje shoda na druhé straně, doplní se **NULL**.

```
SELECT
    film.film_id,
    film.title,
    language.language_id,
    language.name AS language_name
```

```
FROM public.film AS film
FULL OUTER JOIN public.language AS language
  ON language.language_id = film.original_language_id
ORDER BY film.film_id NULLS LAST, language.language_id
LIMIT 20;
```

Filtr v **WHERE** může z výsledku **FULL OUTER JOIN** odstranit jednu nebo obě skupiny neshodných řádků:

```
SELECT
  film.film_id,
  film.title,
  language.language_id,
  language.name AS language_name
FROM public.film AS film
FULL OUTER JOIN public.language AS language
  ON language.language_id = film.original_language_id
WHERE language.name = 'English'
ORDER BY film.film_id NULLS LAST;
```

Druhý dotaz ponechá pouze řádky, kde je jazyk **English**. Filmy bez připojeného jazyka se vyřadí, ale jazyk **English** bez odpovídajícího filmu by zůstal ve výsledku s **NULL** ve sloupcích filmu.

8. Kdy použít **ON** a kdy **WHERE**

Použijte podmínku v **ON**, když:

- chcete zachovat všechny řádky z levé strany **LEFT JOIN**;
- pouze omezujete, které řádky z pravé tabulky se připojí;
- chcete zachovat informaci o tom, že shoda neexistuje pomocí hodnot **NULL**.

Použijte podmínku ve **WHERE**, když:

- chcete filtrovat výsledné řádky až po provedení spojení;
- je žádoucí odstranit i řádky bez shody;
- vědomě chcete, aby se **LEFT JOIN** v daném případě choval jako **INNER JOIN**.

Přehled

Konstrukce	Chování
INNER JOIN ... ON ...	vrátí pouze řádky se shodou
LEFT JOIN ... ON ...	zachová všechny řádky z levé tabulky
RIGHT JOIN ... ON ...	zachová všechny řádky z pravé tabulky
FULL OUTER JOIN ... ON ...	zachová všechny řádky z obou tabulek
podmínka v ON	omezuje vznik shody
podmínka v WHERE	filtruje celý výsledek spojení

Konstrukce**Chování**

```
WHERE pravá_tabulka.sloupec = hodnota po LEFT  
JOIN
```

obvykle odstraní řádky s **NULL** na pravé straně

```
WHERE ... OR pravá_tabulka.sloupec IS NULL
```

zachová i řádky bez shody

Při tvorbě tohoto dokumentu byl využit GitHub Copilot.